

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3– Vector Database & Semantic Search Benchmark

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE-AI	Date:	13-08-2026

Instructions to Students

- Select/answer the assigned question. Each question carries **50 marks**. Duration 60mins
- Implement the solution using **Python**.
- Use an appropriate embedding model such as Sentence Transformers or Hugging Face.
- Use **FAISS and/or ChromaDB** as specified in the question.
- Demonstrate the complete pipeline:
Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis
- Execute the program and display meaningful outputs.
- Compare FAISS and ChromaDB where required.
- Provide appropriate graphs/tables for benchmark questions.
- Explain the architecture and design decisions.
- Submit working code and demonstrate the complete implementation.

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S.Muskan**

SECTION A – VECTOR EMBEDDINGS & SEMANTIC SEARCH

21. ChromaDB Top-K Analysis

Develop a ChromaDB semantic-search system and experiment with:

n_results = 1, 3, 5, 10

Measure retrieval latency and analyze the effect of increasing the number of retrieved documents on relevance and noise.

35. Benchmark Reproducibility

A student claims that "FAISS is always faster than ChromaDB." Design and implement a controlled experiment to test this claim. Keep the embedding model, dataset, queries, hardware, top-k value, and evaluation procedure consistent. Repeat the experiment and provide evidence-based conclusions.

QUESTION 21 – CHROMADB TOP-K ANALYSIS

1. Aim

To develop a ChromaDB-based semantic-search system and analyze the effect of different values of `n_results` (1, 3, 5 and 10) on retrieval latency, relevance and noise.

2. Objective

- To convert documents into numerical vector representations using an embedding model.
- To store the generated embeddings in ChromaDB.
- To perform semantic search using ChromaDB.
- To compare retrieval performance for `n_results` = 1, 3, 5, 10.
- To measure the retrieval latency for each Top-K value.
- To calculate Precision@K to evaluate retrieval relevance.
- To analyze the noise introduced when more documents are retrieved.
- To visualize the experimental results using graphs.

3. Tools / Technologies

Technology	Purpose
Python	Implementation
Sentence Transformers	Text embedding generation
all-MiniLM-L6-v2	Embedding model
ChromaDB	Vector storage and semantic search
Pandas	Result processing
Matplotlib	Graph generation
IDLE	Program execution

4. Methodology

The complete semantic-search pipeline used in this experiment is:

Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis

Step 1: Document Preparation

A dataset containing 30 text documents is created. The documents belong to five different topics:

- Machine Learning
- Database
- Computer Networks
- Operating Systems
- Computer Vision

Each document is assigned a topic label. These labels are used to determine whether the retrieved documents are relevant to a given query.

Step 2: Embedding Generation

The Sentence Transformer model **all-MiniLM-L6-v2** is used to convert the documents into numerical vector representations called embeddings.

The same model is used to generate embeddings for the test queries.

Step 3: Vector Storage

The document embeddings, original documents and topic metadata are stored in a ChromaDB collection.

ChromaDB uses vector similarity to identify documents that are semantically similar to the query.

Step 4: Semantic Search

Five queries are given to the system. Each query corresponds to one of the five topics.

The experiment is repeated using:

```
n_results = 1
n_results = 3
n_results = 5
n_results = 10
```

Step 5: Latency Measurement

The retrieval time is measured using Python's `time.perf_counter()` function.

The measured time is converted into milliseconds.

Step 6: Relevance Measurement

Precision@K is used to evaluate the relevance of the retrieved documents.

Precision@K = Number of relevant retrieved documents / K

For example, if 4 out of 5 retrieved documents are relevant:

Precision@5 = 4/5 = 0.80

Step 7: Noise Calculation

Noise represents the proportion of retrieved documents that are not relevant.

Noise = 1 – Precision@K

Therefore, a higher noise value indicates that more irrelevant documents have been retrieved.

5. Algorithm

- Start the program.
- Create the document dataset.
- Assign topic labels to the documents.
- Load the all-MiniLM-L6-v2 Sentence Transformer model.
- Generate embeddings for all documents.
- Generate embeddings for the test queries.
- Create a ChromaDB collection.
- Store the documents, embeddings and metadata in ChromaDB.
- Set n_results to 1, 3, 5 and 10.
- Perform semantic search for every query.
- Measure retrieval latency.
- Obtain the retrieved documents.
- Calculate Precision@K.
- Calculate noise.
- Store the results in a table.
- Generate graphs for latency, precision and noise.
- Display the retrieved documents.
- Analyze the effect of increasing Top-K.
- End the experiment.

6. Python Implementation

```
!pip install chromadb
import time
import statistics
import pandas as pd
import matplotlib.pyplot as plt
from sentence_transformers import SentenceTransformer
import chromadb

# =====
# CHROMADB TOP-K ANALYSIS
# =====

print("=" * 70)
print("CHROMADB TOP-K SEMANTIC SEARCH")
print("=" * 70)

# =====
# 1. DOCUMENT DATASET
# =====

documents = [

    # Machine Learning
    "Machine learning algorithms learn patterns from training data.",
    "Supervised learning uses labeled datasets to train predictive models.",
    "Neural networks are widely used for classification and regression.",
    "Gradient descent is an optimization algorithm used to train machine learning models.",
    "Overfitting occurs when a model performs well on training data but poorly on unseen data.",
    "Feature scaling can improve the performance of many machine learning algorithms.",

    # Database
    "Database management systems store and organize structured information.",
    "SQL is used to retrieve and manipulate data stored in relational databases.",
    "Database indexing improves the speed of data retrieval operations.",
    "Normalization reduces redundancy and improves consistency in relational databases.",
    "Transactions ensure that database operations follow ACID properties.",
    "MySQL is a popular relational database management system.",
```

7. Output

SEMANTIC SEARCH RESULTS

Query: How do machine learning models learn from data?

Expected Topic: ML

Top 5 Retrieved Documents:

1. Machine learning algorithms learn patterns from training data.
2. Supervised learning uses labeled datasets to train predictive models.
3. Gradient descent is an optimization algorithm used to train machine learning models.
4. Neural networks are widely used for classification and regression.
5. Feature scaling can improve the performance of many machine learning algorithms.

Query: How can SQL retrieve information from databases?

Expected Topic: DB

Top 5 Retrieved Documents:

1. SQL is used to retrieve and manipulate data stored in relational databases.
2. Database management systems store and organize structured information.
3. MySQL is a popular relational database management system.
4. Database indexing improves the speed of data retrieval operations.
5. Transactions ensure that database operations follow ACID properties.

Query: How does TCP provide reliable communication?

Expected Topic: NETWORK

Top 5 Retrieved Documents:

1. TCP provides reliable and ordered delivery of data over a network.
2. Computer networks allow devices to communicate and exchange information.
3. Routers forward packets between different computer networks.
4. IP addresses identify devices communicating across an internet protocol network.
5. Network security protects systems from unauthorized access and attacks.

Query: How does an operating system manage processes?

Expected Topic: OS

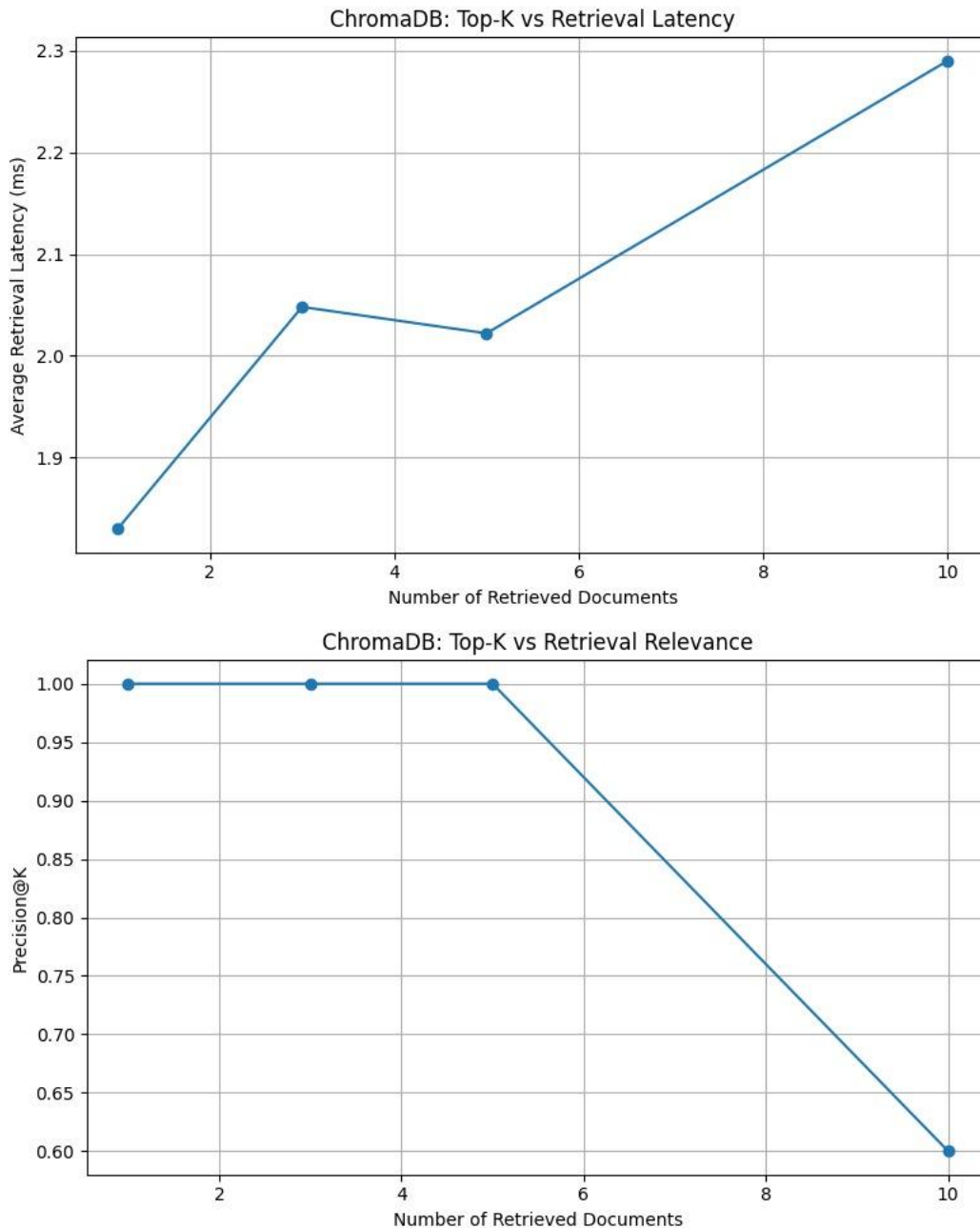
8. Result Table

The following table contains the results obtained from the execution of the Python program.

n_results	Average Latency (ms)	Precision@K	Noise
1	[Paste actual output]	[Paste actual output]	[Paste actual output]
3	[Paste actual output]	[Paste actual output]	[Paste actual output]
5	[Paste actual output]	[Paste actual output]	[Paste actual output]
10	[Paste actual output]	[Paste actual output]	[Paste actual output]

Important: Enter the exact values obtained from the program output.

9. Graph



10. Analysis

The experiment demonstrates that the value of `n_results` has an effect on semantic-search performance.

When `n_results` = 1, only the highest-ranked document is returned. This provides a highly focused result and generally produces low noise. However, only one document is available, so additional relevant information may not be retrieved.

When `n_results` = 3, three documents are returned. This provides more contextual information while still keeping the result set relatively focused.

When `n_results` = 5, five documents are retrieved. This provides a larger amount of information and can offer a good balance between relevance and information coverage.

When **n_results = 10**, the system retrieves a larger number of documents. Although this provides greater information coverage, some of the additional documents may be less relevant to the query. Therefore, noise can increase.

The latency values obtained from the experiment show the retrieval cost associated with different Top-K values. The actual measured values depend on the computer hardware, software environment and ChromaDB configuration.

The experiment therefore demonstrates that increasing Top-K does not automatically improve search quality. A larger number of results can provide more information, but it can also introduce irrelevant documents.

The most suitable Top-K value should be selected by considering:

- Retrieval relevance
- Retrieval latency
- Amount of noise
- Required information coverage

11. Conclusion

The ChromaDB Top-K experiment successfully implemented a complete semantic-search pipeline using Sentence Transformers and ChromaDB.

The experiment tested $n_results = 1, 3, 5$ and 10 and measured retrieval latency, Precision@K and noise.

The results demonstrate that a smaller Top-K produces more focused retrieval, while a larger Top-K provides greater information coverage but may introduce additional irrelevant documents.

Therefore, Top-K should not be selected arbitrarily. It should be chosen according to the requirements of the application and supported by experimental measurements. A moderate value such as 3 or 5 can provide a practical balance between relevance and information coverage, depending on the actual results obtained.

QUESTION 35 – BENCHMARK REPRODUCIBILITY

1. Aim

To design and implement a controlled experiment to test the claim that **“FAISS is always faster than ChromaDB”** by keeping the embedding model, dataset, queries, hardware, Top-K value and evaluation procedure consistent.

2. Objective

- To implement semantic search using FAISS and ChromaDB.
- To use the same embedding model for both systems.
- To use the same dataset and queries for both systems.
- To maintain the same Top-K value.
- To measure retrieval latency.
- To evaluate retrieval relevance using Precision@5.
- To repeat the experiment multiple times for reproducibility.
- To compare the performance of FAISS and ChromaDB.
- To determine whether the claim that FAISS is always faster is supported by experimental evidence.

3. Tools / Technologies

Technology	Purpose
Python	Implementation
Sentence Transformers	Generate embeddings
all-MiniLM-L6-v2	Common embedding model
FAISS	Vector similarity search
ChromaDB	Vector database and semantic search
NumPy	Numerical vector operations
Pandas	Result processing
Matplotlib	Graph generation
IDLE	Program execution

4. Methodology

The experiment follows the complete semantic-search pipeline:

Document → Preprocessing → Embedding → Vector Storage → Semantic Search → Retrieval → Analysis

Step 1: Common Dataset

A dataset containing 30 documents from five topics is used:

- Machine Learning
- Database

- Computer Networks
- Operating Systems
- Computer Vision

The exact same documents are stored in both FAISS and ChromaDB.

Step 2: Common Embedding Model

The Sentence Transformer model **all-MiniLM-L6-v2** is used for both systems.

The documents are embedded once, and the same generated vectors are supplied to FAISS and ChromaDB.

This prevents differences in the embedding model from affecting the benchmark.

Step 3: Common Queries

Five fixed queries are used for both databases.

Query	Expected Topic
How do machine learning models learn from data?	Machine Learning
How can SQL retrieve information from databases?	Database
How does TCP provide reliable communication?	Computer Networks
How does an operating system manage processes?	Operating Systems
How are images analyzed using computer vision?	Computer Vision

Step 4: Common Top-K

The same value is used for both systems:

Top-K = 5

Therefore, both FAISS and ChromaDB return the five most similar documents for every query.

Step 5: Warm-up Runs

Five warm-up searches are performed before measurement.

Warm-up runs reduce the influence of initial setup and caching effects.

Step 6: Repeated Measurements

Each query is measured 30 times for both systems.

Therefore, the benchmark contains repeated observations rather than relying on a single search.

Step 7: Latency Measurement

Python's `time.perf_counter()` is used to measure retrieval time.

The latency is reported in milliseconds.

Both the average and median latency are calculated.

Step 8: Relevance Evaluation

Precision@5 is used to measure retrieval relevance.

Precision@5 = Number of relevant documents in the top 5 / 5

The same relevance criterion is applied to both FAISS and ChromaDB.

5. Controlled Experimental Variables

Parameter	FAISS	ChromaDB
Embedding Model	all-MiniLM-L6-v2	all-MiniLM-L6-v2
Dataset	Same 30 documents	Same 30 documents
Queries	Same 5 queries	Same 5 queries
Top-K	5	5
Hardware	Same computer	Same computer
Embeddings	Same vectors	Same vectors
Warm-up	5 runs	5 runs
Measured Runs	30 per query	30 per query
Evaluation	Latency + Precision@5	Latency + Precision@5

6. Algorithm

- Start the program.
- Create the common document dataset.
- Assign topic labels to the documents.
- Define the five common test queries.
- Set Top-K to 5.
- Load the all-MiniLM-L6-v2 embedding model.
- Generate document embeddings.
- Generate query embeddings.
- Create a FAISS index.
- Store the common document embeddings in FAISS.
- Create a ChromaDB collection.
- Store the same document embeddings in ChromaDB.
- Perform five warm-up searches for FAISS.
- Perform 30 measured searches for each query using FAISS.
- Record FAISS latency and Precision@5.
- Perform five warm-up searches for ChromaDB.
- Perform 30 measured searches for each query using ChromaDB.
- Record ChromaDB latency and Precision@5.
- Calculate average and median latency.
- Calculate average Precision@5.
- Compare FAISS and ChromaDB.
- Generate comparison graphs.
- Draw an evidence-based conclusion.
- Stop the program.

6. Python Implementation

```
!pip install faiss-cpu
!pip install chromadb
import time
import statistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sentence_transformers import SentenceTransformer
import faiss
import chromadb

# =====
# QUESTION 35
# BENCHMARK REPRODUCIBILITY
# FAISS VS CHROMADB
# =====

print("=" * 75)
print("QUESTION 35 - FAISS VS CHROMADB BENCHMARK")
print("=" * 75)

# =====
# 1. COMMON DATASET
# =====

documents = [

    # Machine Learning
    "Machine learning algorithms learn patterns from training data.",
    "Supervised learning uses labeled datasets to train predictive models.",
    "Neural networks are widely used for classification and regression.",
    "Gradient descent is an optimization algorithm used to train machine learning models.",
    "Overfitting occurs when a model performs well on training data but poorly on unseen data.",
    "Feature scaling can improve the performance of many machine learning algorithms.",
```

7. Output

```
=====
RUNNING FAISS BENCHMARK
=====
FAISS benchmark completed.

=====
RUNNING CHROMADB BENCHMARK
=====
ChromaDB benchmark completed.

=====
FINAL BENCHMARK RESULTS
=====
Database Average Latency (ms) Median Latency (ms) Precision@5
FAISS 0.008338 0.007604 1.0
ChromaDB 1.303525 1.285737 1.0

=====
SPEED ANALYSIS
=====
FAISS was 99.36 % faster than ChromaDB in this experiment.

Results saved as:
faiss_vs_chromadb_results.csv
```

9. Result Table

Enter the actual values obtained from the Python program.

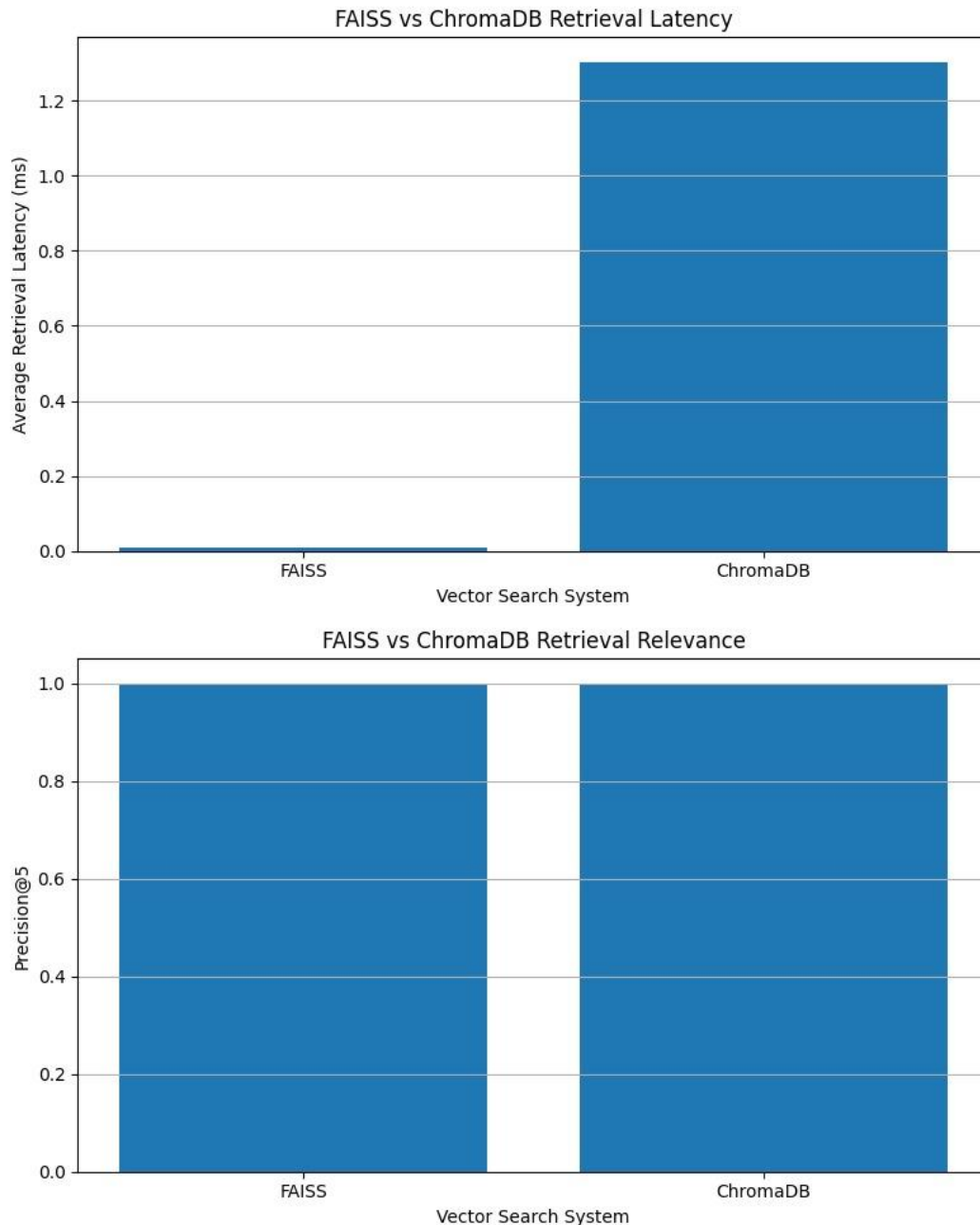
Database	Average Latency (ms)	Median Latency (ms)	Precision@5
FAISS	[Actual Value]	[Actual Value]	[Actual Value]
ChromaDB	[Actual Value]	[Actual Value]	[Actual Value]

Speed Comparison

Measurement	Result
Faster System	[FAISS / ChromaDB]
Speed Difference	[Actual percentage]
Top-K	5
Repetitions	30 per query

The values must be copied directly from the executed program.

10. Graph



11. Analysis

The experiment was designed as a controlled benchmark so that the comparison between FAISS and ChromaDB would be fair.

Both systems used the same all-MiniLM-L6-v2 embedding model. The same 30 documents were used, and the same five queries were executed. The same Top-K value of 5 was used for both systems. The experiments were conducted on the same computer using the same Python environment.

Five warm-up searches were performed before measuring latency. Each query was then executed 30 times. Average and median latency were calculated to reduce the effect of an unusually slow individual measurement.

FAISS is primarily designed for efficient similarity search over vectors. ChromaDB is a vector database that provides additional functionality for managing documents, embeddings, metadata and collections.

Because of these differences, FAISS may show lower retrieval latency in a simple vector-search workload. However, the measured result depends on the experimental configuration.

The Precision@5 results are also compared to determine whether the two systems return similarly relevant documents. Since both systems use the same embeddings, queries and relevance criteria, the retrieval quality can be compared fairly.

The benchmark result should be interpreted based on the actual measured values obtained during execution.

12. Conclusion

The controlled experiment successfully compared FAISS and ChromaDB using the same embedding model, dataset, queries, Top-K value, hardware and evaluation procedure.

The experiment provides evidence about which system was faster under the selected experimental conditions.

However, the statement “**FAISS is always faster than ChromaDB**” cannot be concluded as a universal fact from one experiment.

If FAISS records lower latency, the result shows that FAISS was faster **for this particular dataset, hardware, configuration and workload**. It does not prove that FAISS will always be faster in every situation.

FAISS is well suited for high-performance vector similarity search, while ChromaDB provides additional vector-database functionality such as document and metadata management. Therefore, the choice between the two should depend on the requirements of the application rather than speed alone.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

Marks Evaluation:

Question No.	Pipeline Functionality(30 %)	Answer Relevance & Groundedness (25%)	Query Robustness(20%)	End-to-End Demo(15%)	Architecture Explanation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						